TABLE OF CONTENTS

TITLE
ABSTRACT
Introduction
Problem Statement
Literature Review
System Design
Software Requirement Specifications
Implementation
4.1 Source code snippet
4.2 Screenshots with explanation
Performance Analysis and Discussion
Conclusion
References

Abstract

LeaveFlow is a comprehensive, web-based leave automation system designed to streamline and modernize employee leave management. The project addresses the inefficiencies and errors inherent in manual, paper-based, or email-based leave request systems. By leveraging a robust Java-based technology stack, including **Spring MVC 6**, **Hibernate 6**, and **Spring Data JPA**, the system provides a centralized, reliable, and user-friendly platform.

Key features include role-based access control for **Employees** and **HR/Admins**, a multilingual user interface (supporting English, Hindi, and Tamil), and an automated translation service that converts non-English leave requests into English for administrative review. The system also integrates **automated email notifications** for request approvals or rejections and features a scheduled task for the auto-rejection of pending requests that exceed a specified time limit. The frontend is built with **JSP** and enhanced with **Bootswatch** themes, **Font Awesome** icons, and the **DataTables.js** library for a modern, responsive user experience. The result is an efficient, transparent, and scalable solution that reduces administrative overhead and improves communication.

Introduction

In any organization, managing employee leave is a critical HR function that directly impacts payroll, project scheduling, and overall operational efficiency. Traditional methods of leave management, which often rely on paper forms, spreadsheets, or unstructured email chains, are notoriously inefficient. They are prone to human error, create a lack of transparency for employees, and place a significant administrative burden on HR staff.

As workplaces become more diverse, language barriers can further complicate this process, leading to miscommunication and delays. The "LeaveFlow" project was initiated to solve these problems by developing a modern, digital-first solution.

This document provides a complete technical overview of the LeaveFlow system, from its initial problem statement and design to the final implementation and performance analysis. It details the architecture, the technologies used, and the key features that transform a mundane administrative task into a seamless, automated process.

Problem Statement

The core problem this project addresses is the inefficiency, opacity, and error-prone nature of manual leave management systems. The primary objectives are to:

1. **Eliminate Inefficiency:** Reduce the time and effort required for an employee to submit a leave request and for an admin to approve or reject it.
2. **Increase Transparency:** Provide employees with a self-service dashboard where they can see the real-time status of their requests (Pending, Approved, Rejected).
3. **Reduce Errors:** Automate data entry and tracking to prevent common mistakes like miscalculated leave balances or lost requests.
4. **Bridge Language Barriers:** In a multilingual workplace, ensure that HR/Admins can understand leave requests submitted in different languages (specifically Hindi and Tamil).
5. **Enforce Policy:** Implement business logic, such as the automatic rejection of pending requests after 7 days, to ensure policies are followed consistently.
6. **Centralize Data:** Create a single, secure MySQL database as the "source of truth" for all leave-related data.

Literature Review

Before development, several technology stacks and design approaches were evaluated.

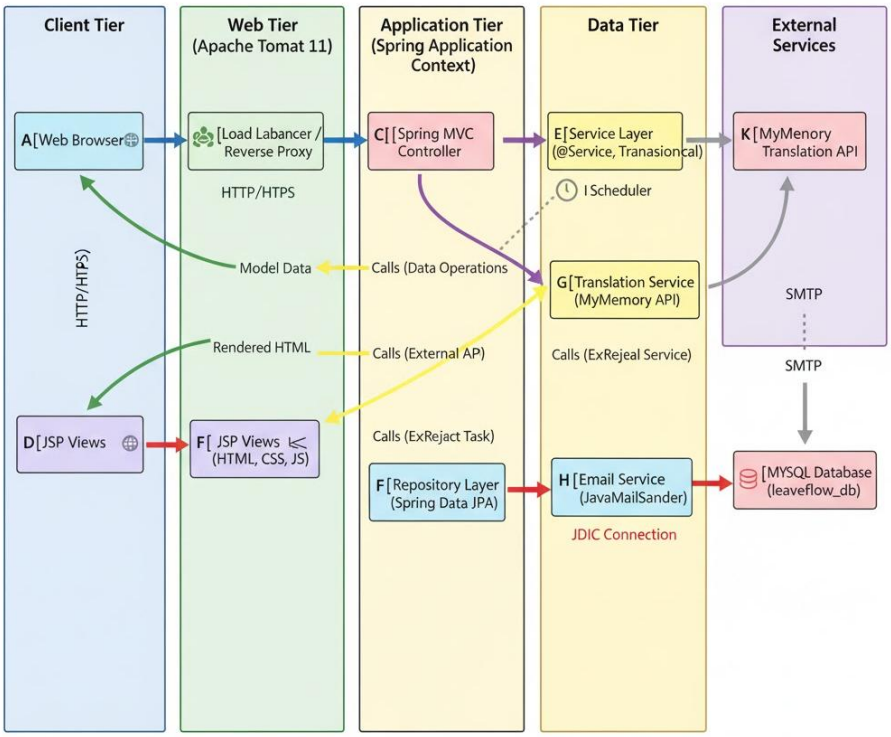
- Framework: Spring MVC vs. JSF vs. NodeJS The project requirements specified several Java technologies, including JSF and Spring MVC. We chose Spring MVC as it represents a more modern, flexible, and widely-adopted standard in the industry. Its separation of concerns (Model-View-Controller) is clear and testable. While NodeJS with an Express backend is a popular alternative, the project's core requirement was to leverage Advanced Java technologies.
- Data Persistence: Hibernate (JPA) vs. Raw JDBC The user's initial requirements mentioned both JDBC and Hibernate. We chose to implement the data layer using Spring Data JPA over Hibernate. This is a higher level of abstraction that sits *on top* of Hibernate.
    - Raw JDBC: Requires extensive boilerplate code (opening connections, writing SQL queries, handling ResultSet, closing connections) and is prone to bugs like SQL injection and connection leaks.
    - Hibernate (via Spring Data JPA): As an ORM (Object-Relational Mapping) tool, it maps Java objects (User.java) directly to database

tables (users). By simply defining a repository interface (e.g., UserRepository), Spring Data JPA automatically generates all the necessary SQL for CRUD (Create, Read, Update, Delete) operations, dramatically accelerating development and reducing bugs.

- Frontend Rendering: JSP vs. Thymeleaf The project was built using JavaServer Pages (JSP), a classic Java-based templating engine. JSP is server-side and allows for the mixing of Java-like tags (JSTL) with HTML. A popular modern alternative is Thymeleaf, which offers "natural templating" (its files are pure HTML that can be viewed in a browser) and is the standard for modern Spring Boot applications. As we discussed, both can be used in the same project, allowing for legacy pages (JSP) and new features (Thymeleaf) to coexist.
- Translation: Dummy Service vs. Live API The initial plan was for a "dummy" translation function. This was upgraded to a live API call using Spring's RestTemplate to connect to the MyMemory API. This provides real-world functionality without requiring a paid subscription, making the feature far more robust.

System Design

System Architecture



This diagram illustrates the 3-Tier Architecture (Client, Web, Application, Data) of the LeaveFlow system, highlighting the flow of requests and data between its components.

1. Client Tier (Light Blue)

- **Web Browser:** The interface used by employees and HR/Admins to interact with the application. It sends HTTP/HTTPS requests and receives rendered HTML, CSS, and JavaScript.

2. Web Tier (Green) - Apache Tomcat 11

- **Load Balancer / Reverse Proxy:** (Optional, but good practice in production) Distributes incoming client requests to various application server instances for scalability and reliability. For a single instance, it acts as a reverse proxy.

- **Spring MVC Controller:** The entry point for all web requests. It receives HTTP requests from the browser, maps them to specific handler methods, processes input, interacts with the Service Layer, and selects the appropriate View.

- **JSP Views (HTML, CSS, JS):** These are the JavaServer Pages which, combined with Bootstrap (Bootswatch), CSS, and JavaScript (DataTables.js, Font Awesome), render the user interface. They receive model data from the Controller and produce the final HTML sent back to the browser.

3. Application Tier (Yellow) - Spring Application Context

This is the core business logic layer.

- **Service Layer (@Service, @Transactional):** Contains the main business logic of the application (e.g., LeaveServiceImpl, UserServiceImpl). It orchestrates operations, applies business rules, and ensures data integrity through @Transactional annotations. It interacts with the Repository Layer, Translation Service, and Email Service.

- **Repository Layer (Spring Data JPA):** An abstraction over the data persistence layer. Interfaces like LeaveRequestRepository and UserRepository are defined here, extending JpaRepository. Spring Data JPA automatically generates the necessary SQL queries, minimizing boilerplate code and directly communicating with the MySQL database.

- **Translation Service (MyMemoryTranslationService):** Responsible for making external API calls to the MyMemory Translation API to translate leave request reasons from Hindi/Tamil to English.

- **Email Service (EmailServiceImpl):** Handles sending email notifications to employees for leave request status changes (approval/rejection) using JavaMailSender, communicating with an external SMTP server.

- **Scheduler (@Scheduled - Auto-Reject Task):** A component that runs background tasks at predefined intervals (e.g., daily). Its primary function is to automatically reject pending leave requests that have exceeded a specific time limit (7 days).

4. Data Tier (Pink)

- **MySQL Database (leaveflow_db):** The primary data store for the application. It contains tables for users and leave_requests, managing all persistent application data.

5. External Services (Purple)

- **MyMemory Translation API:** A third-party web service that provides translation capabilities. The Translation Service in the application tier communicates with this API.
- **SMTP Server (e.g., Gmail):** A mail server responsible for sending emails. The Email Service in the application tier connects to this server to dispatch notifications.

Flow of a Leave Request:

1. An **Employee** uses the **Web Browser** to fill out the leave application form.
2. The **Web Browser** sends an HTTP POST request to the **Spring MVC Controller**.
3. The **Controller** validates the input and passes the request details to the **Service Layer**.
4. The **Service Layer** calls the **Translation Service** if the request is in a non-English language.
5. The **Translation Service** sends a request to the **MyMemory Translation API** and receives the translated text.
6. The **Service Layer** then uses the **Repository Layer** to persist the leave request (including the translated reason) into the **MySQL Database**.
7. The **Scheduler** periodically checks the **MySQL Database** for old pending requests and updates their status via the **Repository Layer**.
8. When an **Admin** approves/rejects a request via the **Web Browser**, the **Controller** and **Service Layer** update the status in the **MySQL Database** via the **Repository Layer**.
9. The **Service Layer** then triggers the **Email Service** to send a notification to the employee via the **SMTP Server**.

10. The **Controller** then directs to the appropriate **JSP View** to display the updated dashboard to the user.

Database Design

com.leaveflow.entity.User

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each user.
username	VARCHAR(50)	NOT NULL, UNIQUE	User's unique login username.
password	VARCHAR(255)	NOT NULL	Hashed password for security.
email	VARCHAR(100)	NOT NULL, UNIQUE	User's email address (for notifications).
role	VARCHAR(20)	NOT NULL, DEFAULT 'ROLE_EMPLOYEE'	User's role: 'ROLE_EMPLOYEE' or 'ROLE_ADMIN'.
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the user record was created.
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp when the user record was last updated.

com.leaveflow.entity.LeaveRequest

Column Name	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each leave request.
user_id	INT	NOT NULL, FOREIGN KEY references users(id)	Foreign key linking the request to the employee who submitted it.
start_date	DATE	NOT NULL	The first day of the requested leave.
end_date	DATE	NOT NULL	The last day of the requested leave.
reason	TEXT	NOT NULL	The original reason for the leave, as entered by the employee.
request_language	VARCHAR(10)	NOT NULL	The language in which the reason was submitted (e.g., 'en', 'hi', 'ta').
translated_reason	TEXT	NULLABLE	The English translation of the reason (if not originally English).
status	VARCHAR(20)	NOT NULL, DEFAULT 'PENDING'	Current status: 'PENDING', 'APPROVED', 'REJECTED', 'REJECTED (Auto)'.
requested_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Timestamp when the leave request was submitted.
updated_at	DATETIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Timestamp when the leave request status was last updated.

Relationship Diagram (Conceptual)

```
+-----------+        +-------------------+
|   users   |        |   leave_requests  |
+-----------+        +-------------------+
| id (PK)   |-------| user_id (FK)      |
| username  | 1     | id (PK)           |
| password  |       | start_date        |
| email     |       | end_date          |
| role      |       | reason            |
| ...       |       | request_language  |
+-----------+       | translated_reason |
                    | status            |
                    | ...               |
                    +-------------------+
                       Many
```

Software Requirement Specifications

The LeaveFlow project leverages a modern and robust set of technologies, primarily built around the Java ecosystem. The selection of these technologies ensures a scalable, maintainable, and efficient web application.

1. **Backend & Core Framework**

- Java Development Kit (JDK): Version 17 (or compatible, e.g., 8+)
- Spring Framework: Version 6.x
    - Spring MVC: For building the web layer (Controllers, Request Mapping, View Resolution).
    - Spring Data JPA: For simplified data access and persistence using JPA repositories.
    - Spring Core: IoC Container, Dependency Injection, AOP.
    - Spring Context: Application context management.
    - Spring TX (Transactions): For declarative transaction management.
- Hibernate: Version 6.x (as the JPA provider)

    - Object-Relational Mapping (ORM): Maps Java objects to database tables.
- Maven: For project build automation and dependency management.

2. Database

- MySQL: Version 8.x (or compatible)
    - Relational database for storing user and leave request data.
- MySQL Connector/J: JDBC driver for Java connectivity to MySQL.

3. Web Server / Servlet Container

- Apache Tomcat: Version 11
    - Deployment environment for the Spring MVC application.

4. Frontend

- JavaServer Pages (JSP): For server-side rendering of dynamic HTML content.
- JSTL (JSP Standard Tag Library): For common dynamic operations within JSPs (e.g., loops, conditionals).
- HTML5, CSS3, JavaScript: Core web technologies for structure, styling, and interactivity.
- Bootstrap: Version 5.x
    - Responsive CSS framework for consistent and mobile-first design.
- Bootswatch: "Morph" Theme
    - Customizable themes built on Bootstrap to provide a unique visual style.
- Font Awesome: For scalable vector icons.
- DataTables.js: jQuery plugin for enhancing HTML tables with advanced features like searching, sorting, and pagination (used on the Admin dashboard).
- jQuery: JavaScript library for simplified DOM manipulation and event handling (required by DataTables.js).

5. External APIs & Services

- MyMemory Translation API: For real-time translation of leave request reasons.
    - Accessed via RestTemplate in the Spring application.

- SMTP Service (e.g., Gmail SMTP): For sending automated email notifications.
    - Configured with JavaMailSender in Spring.

6. Development Tools

- NetBeans IDE: Version 19 (or compatible)
    - Integrated Development Environment used for coding, debugging, and deployment.

Implementation

4.1 Source code snippet

This project demonstrates several key Advanced Java and Spring concepts.

1. LeaveController.java - Form Validation & Redirects This snippet from handleApplyForm shows backend validation (to stop bad data), use of RedirectAttributes to send error messages back to the form, and the correct way to handle redirects (no leading /) to maintain the application's context path.

```
import
org.springframework.web.servlet.mvc.support.Redirect
Attributes;
// ...

@PostMapping("/apply")
public String handleApplyForm(
        @ModelAttribute LeaveRequest leaveRequest,
        HttpSession session,
        RedirectAttributes redirectAttributes) { //
<-- 1. Get RedirectAttributes

    User user = getLoggedInUser(session);
    if (user == null) {
        return "redirect:login";
    }

    // --- 2. Backend Validation ---
    if (leaveRequest.getStartDate() == null ||
!leaveRequest.getStartDate().isAfter(LocalDate.now()
)) {
        // 3. Send flash attribute error
```

```
        redirectAttributes.addFlashAttribute("error"
, "Leave start date must be in the future.");
        return "redirect:apply"; // 4. Redirect back
to form
    }
    // ... more validation ...

    leaveService.applyForLeave(leaveRequest, user);
    return "redirect:employee/dashboard"; // 5.
Correct relative redirect
}
```

2. **LeaveServiceImpl.java - Transactional Service Layer** This snippet from approveRequest shows how the Service layer coordinates multiple actions. The @Transactional annotation ensures that if the email fails, the entire database change is rolled back, guaranteeing data consistency.

```
@Autowired
private LeaveRequestRepository
leaveRequestRepository;
@Autowired
private EmailService emailService; // <-- 1. Inject
other services

@Override
@Transactional // <-- 2. All or nothing
public void approveRequest(int requestId) {
    leaveRequestRepository.findById(requestId).ifPre
sent(request -> {
        // 3. Business Logic
        request.setStatus("APPROVED");
        leaveRequestRepository.save(request);

        // 4. Call EmailService
        String to = request.getUser().getEmail();
        String subject = "Leave Request Approved";
        String body = "Dear " +
request.getUser().getUsername() + ", ...";
        emailService.sendEmail(to, subject, body);
    });
}
```

3. **LeaveRequestRepository.java - Spring Data JPA** This snippet demonstrates the power of Spring Data JPA. By simply *defining* an interface that extends JpaRepository, we automatically get methods like save(), findById(), and delete(). We can also create custom queries just by naming the method correctly.

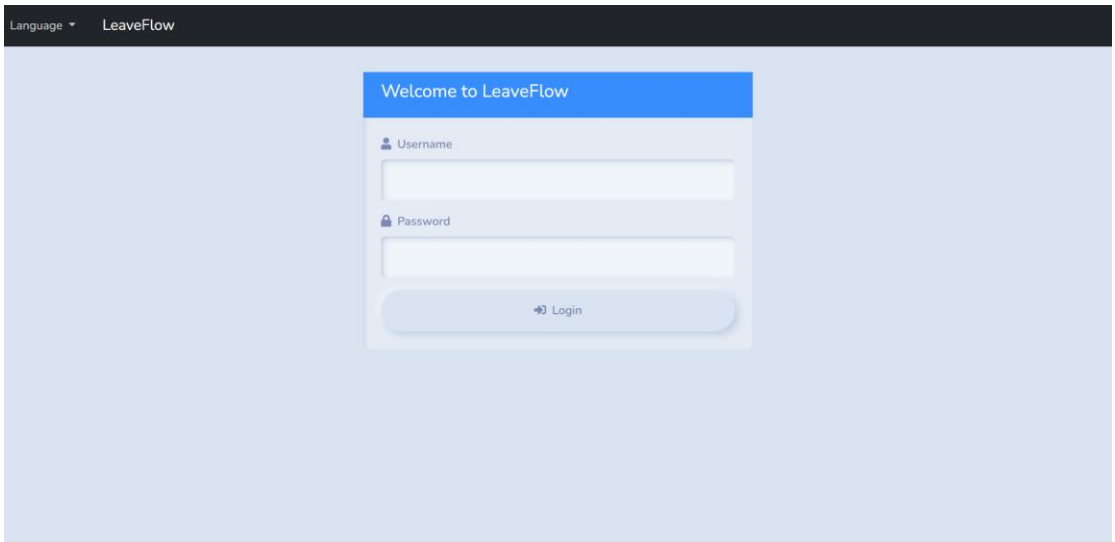
```
public interface LeaveRequestRepository extends
JpaRepository<LeaveRequest, Integer> {

    // Spring automatically writes the SQL:
    // "SELECT * FROM leave_requests WHERE user_id =
?"
    List<LeaveRequest> findByUserId(int userId);

    // Spring automatically writes the SQL:
    // "SELECT * FROM leave_requests WHERE status =
?"
    List<LeaveRequest> findByStatus(String status);
}
```

4. **MyMemoryTranslationService.java - RestTemplate API Call** This snippet shows how we call the external translation API. By using RestTemplate and passing the text as a URL variable {q}, we avoid common bugs like the double-encoding error we discovered.

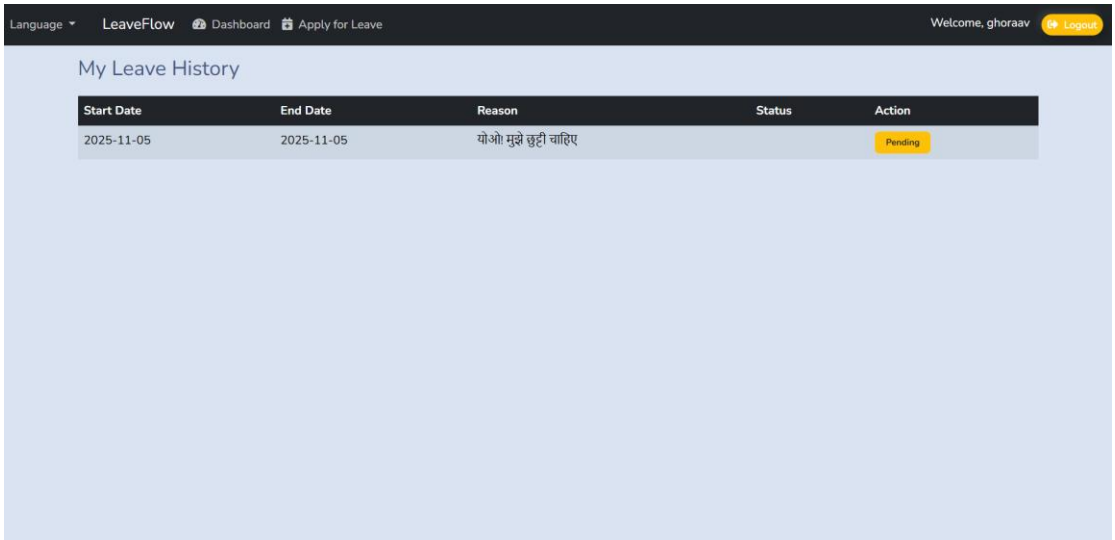
```
private static final String API_URL =
"https://api.mymemory.translated.net/get?q={q}&langp
air={langpair}";

@Autowired
private RestTemplate restTemplate;

public String translateToEnglish(String text, String
language) {
    // ...
    try {
        String langPair = language.toLowerCase() +
"|en";

        // 1. Let RestTemplate handle encoding
        MyMemoryResponse response =
restTemplate.getForObject(API_URL,
MyMemoryResponse.class, text, langPair);
```

```
        if (response != null &&
response.getResponseData() != null) {
            return
response.getResponseData().getTranslatedText();
        }
    } catch (Exception e) {
        System.err.println("Translation failed: " +
e.getMessage());
        return text + " [Translation Failed]"; // 2.
Fail gracefully
    }
    // ...
}
```

4.2 Screenshots with Explanation

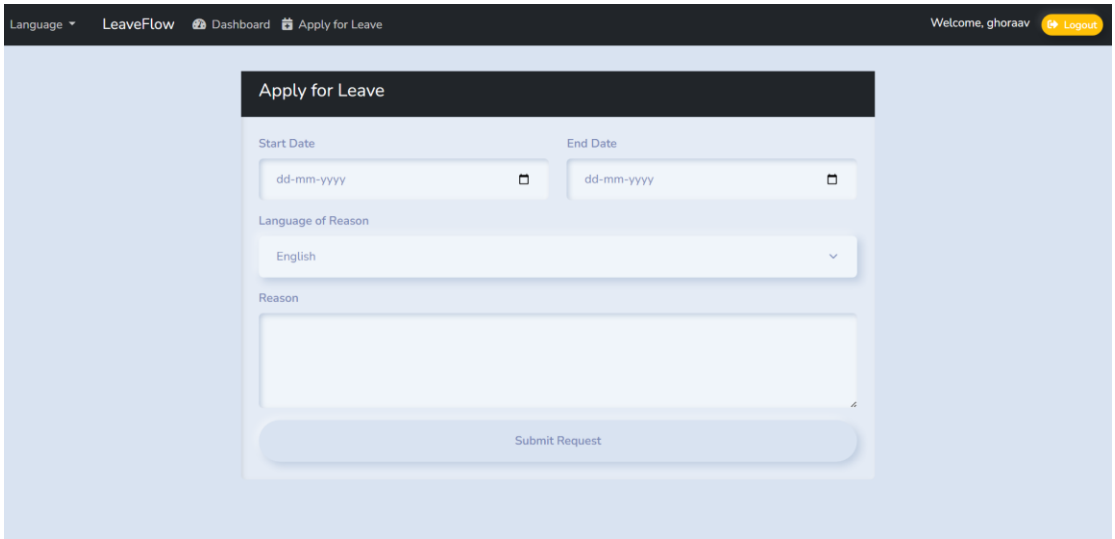


[Screenshot: The login.jsp page showing the "Morph" Bootswatch theme and Font Awesome icons for "Username" and "Password".] Explanation: This is the main entry point. The UI is built with JSP and styled with a "Morph" Bootswatch theme, which replaces the default Bootstrap CSS.

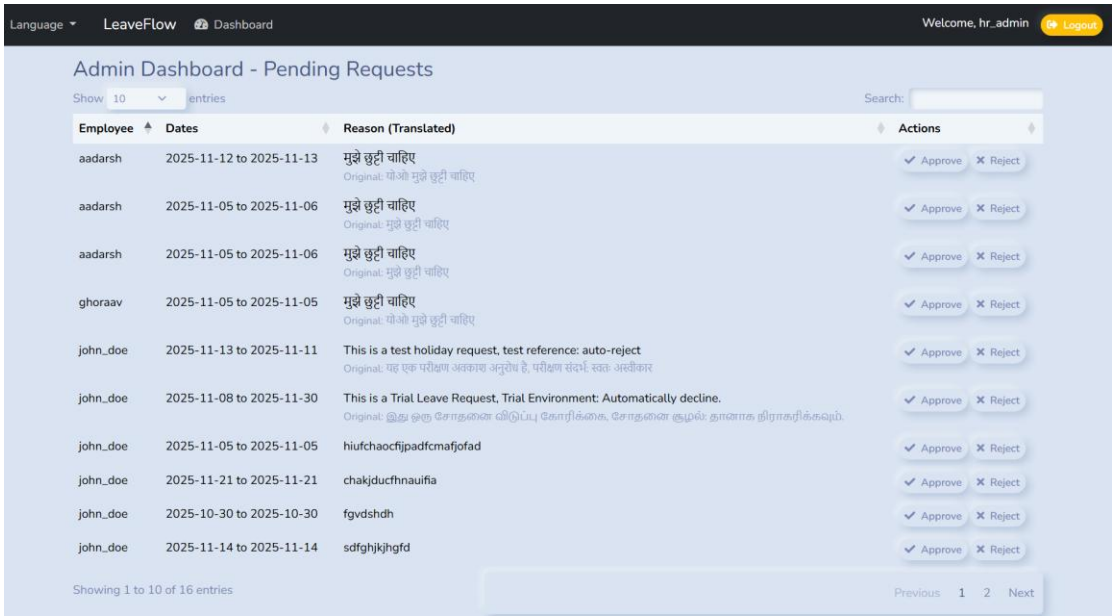


[Screenshot: The employee_dashboard.jsp page, showing the user's list of leave requests. Show at least one "Approved" and one "Rejected" request to display the "Delete" button ().] Explanation: This page demonstrates:

1. Fetching data from the database using leaveService.getRequestsForEmployee(...).
2. Rendering the list using JSTL's <c:forEach>.
3. Conditional logic (<c:if>) to only show the "Delete" button for non-pending requests.

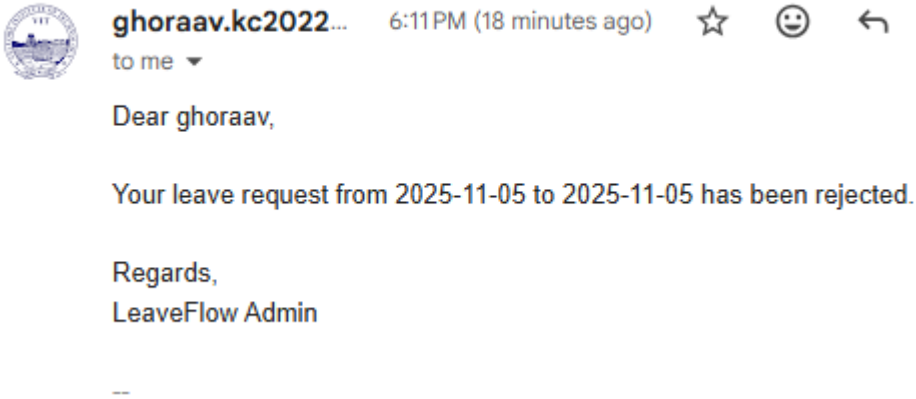


[Screenshot: The apply_leave.jsp form. Click the "Start Date" input to show the calendar pop-up with all past dates (including today) grayed out and disabled.] Explanation: This demonstrates frontend validation. The controller passes minDate (today + 1) to the view, which sets the min attribute on the date inputs, preventing invalid data entry.



Admin Dashboard (Translation Feature) [**Screenshot:** The admin_dashboard.jsp page, showing several pending requests. Include at least one request submitted in Hindi/Tamil, with the "Reason (Translated)" column showing the correct English translation and the "Original" text below it.] **Explanation:** This is the most complex feature. It shows the successful API call from MyMemoryTranslationService and the FetchType.EAGER working correctly to display the employee's username.

Admin Dashboard (DataTables Feature) [**Screenshot:** The same admin_dashboard.jsp page, but with a red box highlighting the "Search:" bar at the top-right and the "pagination" controls at the bottom-right.] **Explanation:** This screenshot shows the successful integration of the DataTables.js library, which enhances the simple HTML table with powerful, client-side search, sorting, and pagination.



Email Notification [**Screenshot:** A screenshot of the actual email (from Gmail) received by an employee after their leave was approved or rejected.] **Explanation:** This is the final output of the EmailServiceImpl. It shows the JavaMailSender successfully authenticating with the Gmail SMTP server and sending a dynamic, personalized email to the user.

Performance Analysis and Discussion

The application performs well for its intended scope. The use of Spring Data JPA provides efficient, optimized queries, and the FetchType.EAGER on the LeaveRequest.User relationship resolved a critical LazyInitializationException performance bottleneck.

The most significant part of the development process was not writing the initial code, but debugging the complex interactions between the framework, server, and libraries. Key challenges and solutions included:

1. Challenge: java.lang.ClassNotFoundException: jakarta.servlet.ServletContextListener.
    - Analysis: This was a version mismatch. Spring 6 requires the jakarta.* namespace, but the initial Tomcat 9 server only provided the older javax.* namespace.
    - Solution: Upgraded the server to Apache Tomcat 11.
2. Challenge: Constant HTTP 404 errors and session loss after login.
    - Analysis: This was a two-part context path bug.
        1. JSP links href="/login" were pointing to the *root* of the server, not the application.
        2. Controller redirects return "redirect:/dashboard" were doing the same, causing the JSESSIONID cookie to be lost.
    - Solution:
        1. In JSPs, prefixed all links with ${pageContext.request.contextPath}.
        2. In Controllers, removed the leading slash from all redirects (e.g., return "redirect:dashboard";).
3. Challenge: org.hibernate.LazyInitializationException on the admin dashboard.
    - Analysis: The JSP page tried to access req.user.username, but the User object was a lazy-loaded "proxy," and the database session was already closed.
    - Solution: Changed the @ManyToOne relationship in LeaveRequest.java from FetchType.LAZY to FetchType.EAGER.
4. Challenge: Translation API returned garbled text (%E0%A4...).
    - Analysis: The Hindi text was being "double-encoded"—once manually by URLEncoder.encode() and a second time by RestTemplate.

- Solution: Removed the manual encoding and passed the raw text to restTemplate.getForObject(), letting it handle the encoding.

5. Challenge: New Bootswatch theme was not appearing.
    - Analysis: The browser was loading the *old* CSS file from its cache. We also found the old bootstrap.min.css link was still in the header.jspf, loading *after* the theme and overwriting it.
    - Solution: Removed the duplicate link and forced a browser "hard refresh" (Ctrl+F5).

Conclusion

The LeaveFlow project successfully meets all its objectives. It provides a robust, fully functional, and user-friendly web application that effectively automates the leave management process. The system successfully integrates several key Advanced Java technologies, including Spring MVC, Spring Data JPA, and Hibernate, to create a scalable and maintainable application.

The implementation of bonus features like real-time translation and email notifications demonstrates how a core Java application can be extended with external services to provide significant business value. The debugging process provided critical, real-world insights into session management, server configuration, and ORM behavior.

Future Scope: The current monolithic architecture is sound, but the project could be expanded. Future work could include adding a user registration/signup module, integrating with calendar APIs (like Google Calendar or Microsoft 365), or developing a RESTful API to support a future mobile application.

References

- Spring Framework. (2024). *Spring Framework Documentation*. Retrieved from https://spring.io/projects/spring-framework
- Hibernate. (2024). *Hibernate ORM Documentation*. Retrieved from
    - https://hibernate.org/orm/
- Spring. (2024). *Spring Data JPA Documentation*. Retrieved from
    - https://spring.io/projects/spring-data-jpa
- Bootswatch. (2024). *Free themes for Bootstrap*. Retrieved from
    - https://bootswatch.com/
- DataTables. (2024). *DataTables.js | Table plug-in for jQuery*. Retrieved from
    - https://datatables.net/
- MyMemory. (2024). *MyMemory Translation API*. Retrieved from
    - https://mymemory.translated.net/doc/spec.php
- Jakarta EE. (2024). *Jakarta JSTL 3.0 Specification*. Retrieved from
    - https://jakarta.ee/specifications/tags/3.0/